

Diagramme d'activités

- Répond à la question « Qu'est-ce qui se passe? »

Départ/arrivée

Nœud de départ  voir plusieurs nœuds de départ.

Nœud de fin  voir plusieurs nœuds d'arrivée.

- Les diagrammes d'activités permettent de mettre l'accent sur les traitements. Ils sont donc particulièrement adaptés à la modélisation du cheminement de flots de contrôle et de flots de données. Ils permettent ainsi de représenter graphiquement le comportement d'une méthode ou le déroulement d'un cas d'utilisation.

- La différence principale entre les diagrammes d'interaction et les diagrammes d'activités est que les premiers mettent l'accent sur le flot de contrôle d'un objet à l'autre, tandis que les seconds insistent sur le flot de contrôle d'une activité à l'autre.

Action/Activité

Action :

- Une seule opération/étape. Ne peut pas être découpée en plus petit.
- Modélisée par un rectangle aux coins arrondis, avec un titre.



Activité :

- Plusieurs opérations/étapes. Peut être découpée en plus petit.
- Peut être représentée par son propre diagramme d'activités.
- Même modélisation qu'une action.



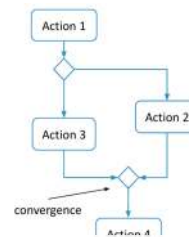
Transition

- Passage d'une action à une autre



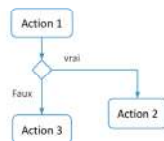
Convergence

- Modélisée par un losange avec plusieurs flux d'entrée, un seul flux de sortie.
- Endroit où deux chemins d'exécution mutuellement exclusifs se rejoignent.
- A partir du point de convergence, la suite du trajet est mutuelle.



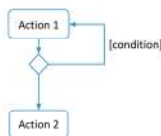
Décision (branche)

- Transition conditionnée.
- Modélisée par un losange (diamant).
- Un seul flux entrant, plusieurs flux sortants.
- Conditions booléennes (vrai/faux) ou autres (exemple: expression de garde [n < 2] ...)



Itération

- Boucle sur une ou plusieurs actions.



Fin locale

- Fin d'une branche uniquement et non pas tout le processus.



Synchronisation: Fork/Join

- Parallélisme entre les activités. Aucun ordre d'exécution.
- Modélisation: barre pleine horizontale ou verticale.
- Seule condition: l'action après le Join ne peut s'exécuter que si les toutes les précédentes sont terminées.

Fork (parallélisme) :

Join (synchronisation)

- Un flux entrant plusieurs flux sortants. Plusieurs flux entrants un flux sortant.



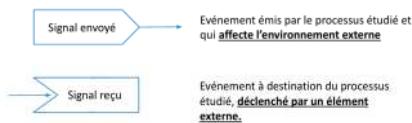
Flots d'objet

- Objets produits ou consommés par une action.



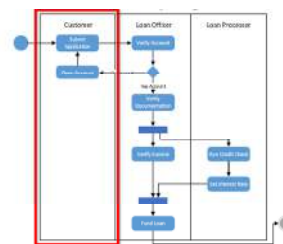
Événements

- Permettent de déclencher un traitement en dehors du processus en cours de modélisation.



Partitions/couloirs d'activité (swimlanes)

- Permettent de montrer les responsabilités dans un processus.
- Chaque activité est tracée dans le couloir de la ressource (système, rôle...) concernée.



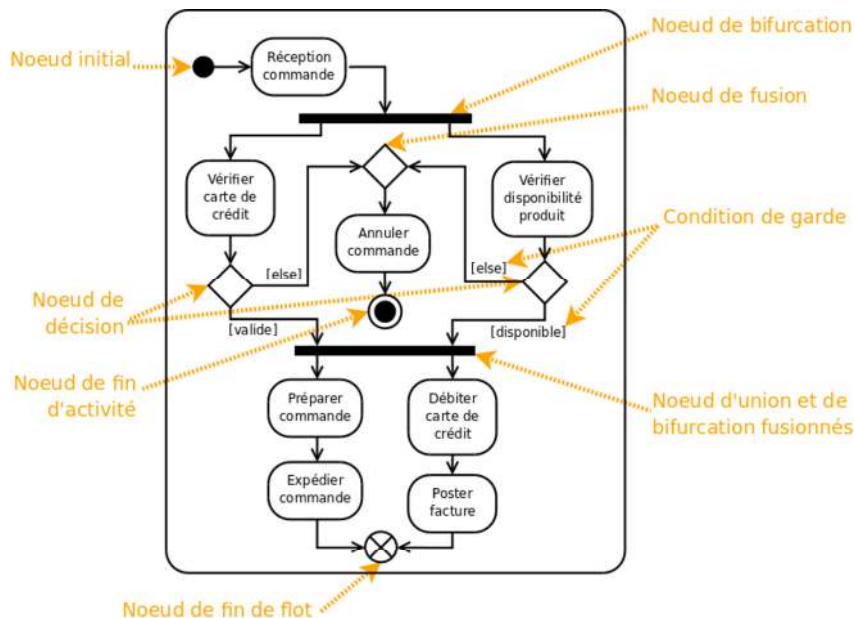


Diagramme Communication

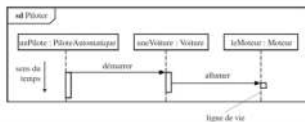


Diagramme de séquence

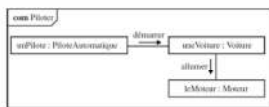


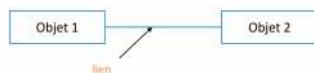
Diagramme de communication

Répond à la question « **Comment** »

Utilisé par : BA, Architecte, Développeur.

- Diagramme dynamique (**interactions**).
- Modélise les interactions entre des objets pour réaliser une action.
- Créé en phase de conception.
- Même principe que le diagramme de séquence, mais avec une représentation spatiale et non temporelle.
- Appelé également diagramme de collaboration (anciennes versions UML).
- Chaque action du diagramme d'activités peut être détaillée dans un diagramme de communication

- Lien (connecteur)** : connexion entre deux objets.



Message :

- Déclenche l'exécution d'une opération de l'objet destinataire.
- On peut avoir plusieurs messages sur un même lien.



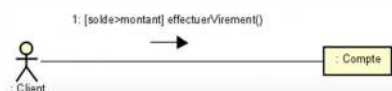
- Types de messages** : synchrone: asynchrone: retour: →
- Syntaxe** : [prédécesseurs/][garde][séquence][itération][valeurRetour:=]signatureMessage(arguments)

Message – Séquence (numérotation) :

- Remplace l'aspect temporel du diagramme de séquence -> ordre d'exécution des messages.
- Numérotation simple: 1, 2, 3,
- Séquencement : 1, 1.1, 1.2, 1.3 : Le point 1 ne peut se terminer que si ses sous-points 1.1, 1.2, 1.3 se terminent.
- Concurrence (parallélisme): 1, 1.1a, 1.1b, 1.2, 1.3: les points 1.1a et 1.1b s'exécutent en //

Message – gardes :

- Condition d'envoi du message : [condition]



Message – itérations :

- Envoi du message autant de fois que le nombre d'itérations : [* itération]



Message – arguments :

- Paramètres de la méthode.



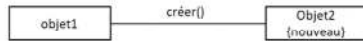
Message – Résultat :

- Valeur retournée par le message.



Message – créer et détruire:

- On ajoute une contrainte associée.
- créer() : ajouter la contrainte {nouveau}



- Détruire() : ajouter la contrainte {détruit}

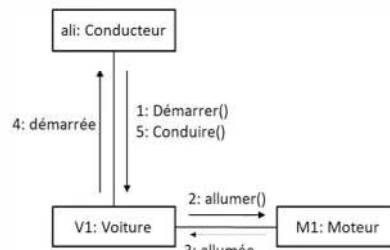


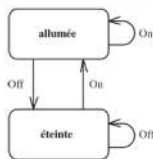
Diagramme d'états-transitions

Représente les traitements et l'enchaînement des états de classe.

Créé pour des objets ayant des comportements dynamiques significatifs.

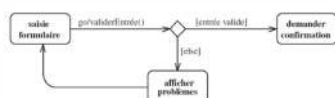
Utilisé par : Business Analyst, Architecte, Développeur.

- Objectif: décrire le comportement dynamique d'une entité (logiciel, composant, ...).
- Permet d'avoir une vue synthétique de la dynamique de l'entité.
- Permet de regrouper un ensemble de scénarios.
- Chaque diagramme d'état concerne un seul objet.
- L'objet passe par des états.
- La transformation d'un état à un autre -> transition.
- Des événements vont initier les transformations.
- Un objet peut passer par plusieurs états durant sa vie.
- Etat** = période de la vie d'un objet durant laquelle il accomplit une activité ou attend un événement.
- Transition: lorsque qu'un objet passe d'un état 1 à un état 2.



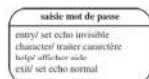
Point de choix :

- Représentation : ◇
- Les gardes après le point de choix sont évaluées au moment où il est atteint.



Transition interne :

- Ne modifie pas l'état courant de l'objet.
- Trois déclencheurs prédéfinis:
 - entry/** activité à effectuer à chaque fois que l'on rentre dans l'état considéré.
 - do/** activité continue qui est réalisée tant que l'on se trouve dans l'état.
 - exit/** activité à effectuer quand on quitte l'état.

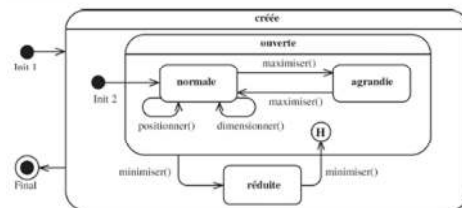


Concepts

Etat initial ● pseudo-état qui définit le point de départ par défaut pour l'automate ou le sous-état.

Etat final ● état qui indique que l'exécution de l'automate ou du sous-état est terminée.

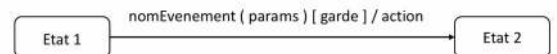
Etat simple



Transition et événement:

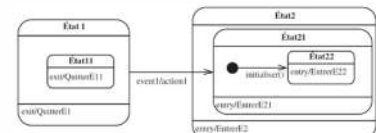
Une **transition** lie deux états. Elle est représentée par une flèche →

Elle indique qu'un **objet** va passer d'un **état** à un **autre** pour exécuter certaines activités **si un événement déclencheur se produit** et que **les conditions de garde sont vérifiées**.



Etat composite :

- Décomposé en deux ou plusieurs sous états (pas de limite de profondeur).



Quand l'événement **event1** survient, l'objet va **QuitterE11** puis **QuitterE1** puis **EntrerE2**, ensuite **EntrerE21**, **initialiser()** et enfin **EntrerE22**.